

```

keep_going = True
iterations = 0
while(keep_going):
    rate_change = step_size * revenue_derivative(current_rate)
    current_rate = current_rate + rate_change

    if(abs(rate_change) < threshold):
        keep_going = False

    if(iterations >= maximum_iterations):
        keep_going = False

    iterations = iterations+1

```

[Listing 3-1](#): Implementing gradient ascent

After running this code, you'll find that the revenue-maximizing tax rate is about 0.528. What we've done in [Listing 3-1](#) is something called *gradient ascent*. It's called that because it's used to ascend to a maximum, and it determines the direction of movement by taking the gradient. (In a two-dimensional case like ours, a gradient is simply called a derivative.) We can write out a full list of the steps we followed here, including a description of our stopping criteria:

1. Start with a `current_rate` and a `step_size`.
2. Calculate the derivative of the function you are trying to maximize at the `current_rate`.
3. Add `step_size * revenue_derivative(current_rate)` to the current rate, to get a new `current_rate`.
4. Repeat steps 2 and 3 until you are so close to the maximum that your current tax rate is changing less than a very small threshold at each step, or until you have reached a number of iterations that is sufficiently high.

Our process can be written out simply, with only four steps. Though humble in appearance and simple in concept, gradient ascent is an algorithm, just like the algorithms described in previous chapters. Unlike most of those algorithms, though, gradient ascent is in common use today and is a key part of many of the advanced machine learning methods that professionals use daily.